

◆ WEB3 SMART CONTRACT SECURITY REVIEW

AVERAGING OVERFLOW

Solidity · Formal verification ·
Halmos

Manual review + working proof-of-concept +
severity-rated report

● Medium 1



GILLES MUSY

Smart contract security researcher

Confirmed findings · Code4rena GiMu84 · Cantina GiLMu

1 Disclaimer

This document is a demonstration security review performed on `src/Average.sol`, a deliberately vulnerable sample authored by Gilles Musy for portfolio purposes. It is not a client engagement: the code was never deployed, holds no user funds, and the vulnerabilities it contains were intentionally introduced to illustrate review methodology, severity classification and proof-of-concept construction. The findings, proofs of concept and remediation guidance below are genuine analysis of the code as written.

A security review is a time-, resource- and expertise-bound effort to surface as many issues as possible; it cannot prove the absence of vulnerabilities. Subsequent reviews, a bug-bounty program and runtime monitoring are recommended for any production deployment.

2 Risk classification

Severity is derived from the impact of an issue and the likelihood of its occurrence.

IMPACT ▼ / LIKELIHOOD ►	HIGH	MEDIUM	LOW
HIGH	Critical	High	Medium
MEDIUM	High	Medium	Low
LOW	Medium	Low	Low

IMPACT High: significant loss of assets or harm to many users. Medium: moderate loss or harm. Low: minor loss or harm.

LIKELIHOOD High: practical under realistic on-chain conditions at low cost. Medium: conditionally incentivised but plausible. Low: needs unlikely assumptions or significant attacker stake.

3 About the target

`src/Average.sol` , a `uint256` averaging library.

TYPE Correctness / integer overflow, demonstrated by formal verification

METHOD Symbolic execution with Halmos over the full `uint256` x `uint256` input space

4 Findings

MEDIUM M-01: Averaging overflow produces an out-of-bounds result (Medium)

ROOT CAUSE

`avg` forms the sum before halving, inside an `unchecked` block:

```
function avg(uint256 a, uint256 b) internal pure returns (uint256) {
    unchecked {
        return (a + b) / 2;
    }
}
```

For inputs whose sum exceeds `type(uint256).max`, `a + b` wraps modulo 2^{256} . The halved result then has no relation to the true mathematical average and can sit entirely outside `[min(a, b), max(a, b)]`. The `unchecked` block is what makes the defect silent: without it the addition would revert, turning a wrong answer into a denial of the call.

THE PROPERTY

The bound is the specification. Stated as a Halmos check over symbolic inputs:

```
function check_avg_withinBounds(uint256 a, uint256 b) public pure {
    uint256 m = Average.avg(a, b);
    uint256 lo = a < b ? a : b;
    uint256 hi = a < b ? b : a;
    assert(lo ≤ m && m ≤ hi);
}
```

COUNTEREXAMPLE (MASTER)

Halmos refutes the property in 0.24s with:

```
a = 0xc000000000000000000000000000000000000000000000000000000000000000
b = 0x4000000000000000000000000000000000000000000000000000000000000000
```

These are $0.75 * 2^{256}$ and $0.25 * 2^{256}$. Their sum is exactly 2^{256} , which wraps to `0`, so `avg(a, b)` returns `0` while `min(a, b)` is $0.25 * 2^{256}$. The result is below both inputs. Any consumer that treats the average as a bounded quantity, a price midpoint, a liquidation threshold, a median feed, is handed a value the rest of its logic was never written to expect.

RECOMMENDATION

Do not form the sum. Order the inputs and add half the gap to the smaller one:

```
function avg(uint256 a, uint256 b) internal pure returns (uint256) {
    (uint256 lo, uint256 hi) = a < b ? (a, b) : (b, a);
    return lo + (hi - lo) / 2;
}
```

`hi - lo` cannot underflow because `lo ≤ hi`, and `lo + (hi - lo) / 2 ≤ hi`, so no intermediate value leaves the `uint256` range. On the `fixed` branch Halmos reports `[PASS]`, proving `lo ≤ m ≤ hi` for all inputs.

SEVERITY

Medium. The bug is a correctness violation in a utility function rather than a direct loss of funds in isolation. Its downstream impact is set by the caller: a contract that relies on the average being

bounded can be driven to a wrong branch or a zero value, which becomes high severity in a pricing or liquidation path and negligible in a purely informational one. Rated as a proven correctness break with context-dependent consequence.

Why a symbolic proof

A fuzzer draws sample inputs and checks them; it can report success while the failing inputs are ones it never generated. The overflow here lives in a thin slice of the 2^{512} input pairs, the ones whose sum crosses 2^{256} . Halmos does not sample. It treats `a` and `b` as symbolic bitvectors and asks an SMT solver whether any assignment violates the assertion. On `master` the answer is yes, with witnesses; on `fixed` the answer is no, which is a proof of the bound across the entire domain. The value of formal verification is precisely that closing statement: not "no failure was observed" but "no failure exists."

Scope and disclaimer

`Average.sol` is intentionally vulnerable demonstration code written to exercise a formal-verification workflow end to end. It is not a production library and not a real client engagement. The single finding above is a genuine correctness bug in this demo, reproduced and then refuted by machine, not an inflated claim.